

Porting the Autoware Safety Island onto nano-ros: a Reference-Consumer Migration Study

NEWSLab, National Taiwan University

Technical report for the Autoware Foundation — 2026-08-20 (rev. 2)

Abstract. The Autoware Safety Island (ASI) runs Autoware’s trajectory follower — MPC lateral and PID longitudinal control — as a standalone application on a safety-class Arm processor, exchanging commands with a full Autoware stack over DDS. This report documents the migration of ASI’s Zephyr targets from a hand-glued CycloneDDS port onto *nano-ros*, a lightweight `no_std` ROS 2 client library for embedded RTOSes. We show how nano-ros’s workspace concepts (component node, bringup, and generated entry packages) restructure the firmware, what changed in the repository by category, how the Arm FVP integration works, and the outcome of a modernization step that advanced the middleware pin by $\approx 2,400$ commits: eleven “walls” with root causes, two upstream issues, and full re-validation — six build modes, a five-phase emulator runtime suite, and a closed-loop demonstration against unmodified ROS 2 Humble Autoware at pre-migration control rates. ASI serves as nano-ros’s canonical *reference consumer*: every integration failure is fixed upstream or tracked, never papered over downstream.

1 Introduction

Safety architectures for autonomous driving separate a *safety island* — a minimal, certifiable control path on a lockstep-capable processor — from the performance-oriented compute domain running the full autonomy stack. ASI realizes this pattern for Autoware: the trajectory follower (MPC lateral and PID longitudinal controllers, vendored unmodified from Autoware) runs on an Arm Cortex-R82 target under Zephyr RTOS, consumes the planned trajectory and vehicle state over DDS, and returns `autoware_control_msgs/Control` commands (optionally mirrored onto CAN).

The original port owned its middleware: a vendored CycloneDDS tree, host IDL tooling, and a bespoke node/executor shim — exactly the code a safety argument least wants to own. nano-ros (a `no_std` ROS 2 client for Zephyr, FreeRTOS, NuttX and ThreadX with interchangeable RMW backends) replaces that glue with a maintained, wire-compatible middleware layer. The migration was run as a *reference-consumer* engagement: ASI is nano-ros’s named consumer archetype, and every consumer-surfaced defect is intaken upstream rather than worked around silently.

2 System overview

Figure 1 shows the validated closed loop. An unmodified Autoware container (ROS 2 Humble, `rmw_cyclonedds`) runs the planning simulator on DDS domain 1; a DDS bridge relays the five controller inputs to domain 2, which is pinned to a host TAP interface; the island on the Arm FVP consumes them and publishes control commands back through the same path. There is no type adaptation anywhere: nano-ros pins its CycloneDDS fork to the 0.10.5 release ROS Humble ships and emits `rmw_cyclonedds`-compatible type names and CDR, so interoperability holds by construction (verified in both directions with `ros2 topic hz / echo`).

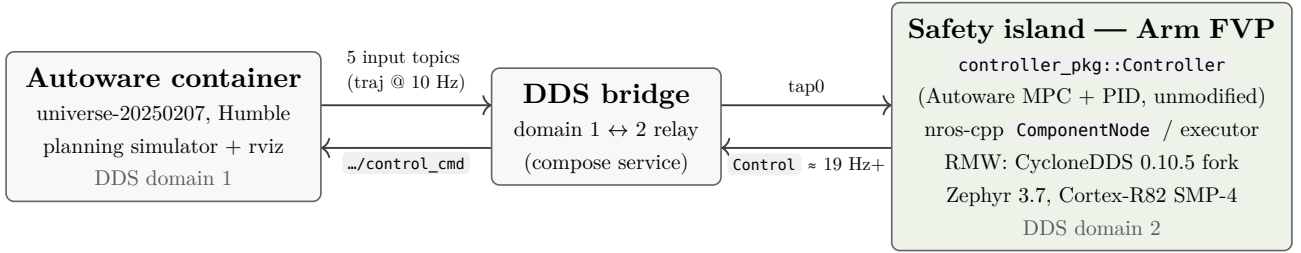


Figure 1: The closed-loop demo (`scripts/run-tap-demo.sh`). The island’s smsc-91c111 NIC model attaches to the host TAP; both DDS sides run stock CycloneDDS wire format.

3 nano-ros concepts in the firmware

nano-ros structures an application as three roles, and the port maps ASI onto them directly:

- **Node package** — `actuation_module/src/controller_pkg/`: the controller as a *component*. `controller_pkg::Controller` derives from the vendored Autoware controller (now a plain implementation library) so the package directory and registered class satisfy nano-ros’s `<pkg-dir>::<Class>` identity rule; `NROS_COMPONENT(Controller)` registers it with an rclcpp shape marker. The base class uses the rclcpp-faithful `nros::ComponentNode` surface: `declare_parameter` (150+ MPC/PID parameters), member-function-pointer `create_subscription / create_timer`, value-typed publishers, and KEEP_LAST-1 QoS on every input.
- **Bringup package** — `actuation_module/src/controller_bringup/`: language-agnostic topology. `system.toml` + `launch/system.launch.xml` declare the node, its remappings and its 74 launch parameters; `play_launch resolve` turns them into a *SystemModel* (a fully resolved, reproducible description with pinned inputs).
- **Entry package** — generated, never hand-written. One CMake call bakes the bootable image from the model (Figure 2); the generated `main` seeds parameters, constructs components by placement-new with executor handles, waits on the board’s network hook, and spins. The retired imperative `main.cpp` is fully superseded — application liveness markers moved into the component constructor as a consequence.

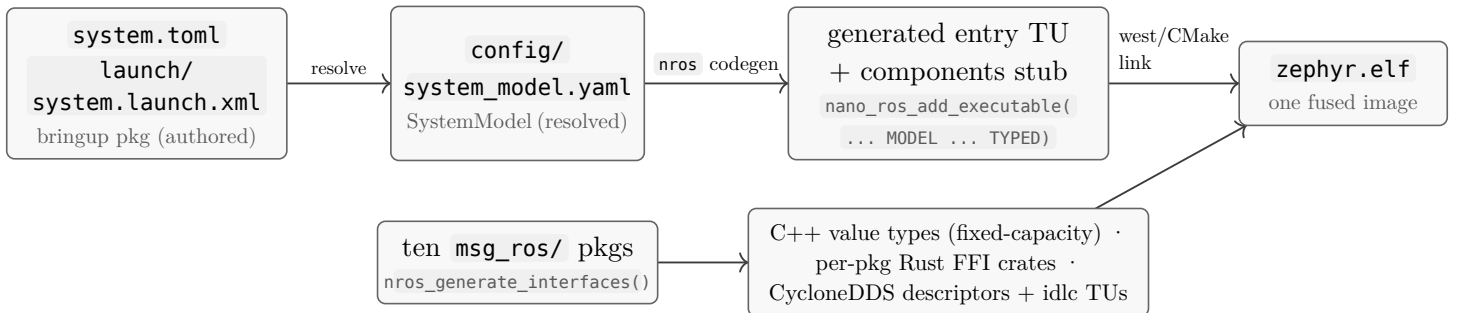


Figure 2: Declarative bringup to bootable image. The launch topology and the interface packages are data; everything executable is generated.

4 What changed: organization and code

The port left the vendored Autoware algorithms untouched and concentrated change in the integration layers. By category, with representative examples:

Category	Organization	Representative changes
Dependencies	nano-ros as git submodule <code>modules/nros</code> , lock-step with the <code>west.yml</code> revision; vendored CycloneDDS retired from the Zephyr path (FreeRTOS targets untouched)	<code>.gitmodules</code> , <code>actuation_module/west.yml</code> (zephyr v3.7.0 + zephyr-lang-rust pinned per nano-ros’s contract)

Category	Organization	Representative changes
Build glue	<code>build.sh</code> drives the nros CLI (host build, codegen tool handoff, board-id mapping, FVP flags); consumer patches are idempotent scripts	<code>-D_NANO_ROS_CODEGEN_TOOL=...</code> , <code>NROS_MAX_PARAMETERS=256</code> / <code>NROS_EXECUTOR_MAX_CBS=16</code> / <code>NROS_SUBSCRIPTION_BUFFER_SIZE=16384</code> , <code>scripts/patches/nros-cpp-embedded-alloc-patch.sh</code>
Node layer	controller re-homed as component pkg <code>controller_pkg/</code> ; polling shim <code>common/node/node_nros.hpp</code> retained for test images only	<code>NROS_COMPONENT(Controller)</code> ; ctor-owned boot markers; one-time <code>nros::init()</code> in the shim; cooperative stop flag replacing <code>pthread_cancel</code>
Messages	ten ROS interface pkgs vendored as <code>.msg</code> sources; a dual-mode umbrella maps ASI aliases onto generated C++ types	<code>autoware_msgs/messages.hpp</code> (<code>TrajectoryMsg_Raw</code> , sentinel <code>_desc</code> stubs); fixed-capacity sequences (250-point trajectory bound)
Tests	five on-target programs kept middleware-honest through the umbrella	<code>unit_test.cpp</code> , <code>dds_loopback_test.cpp</code> on typed pub/sub; <code>can_output_test.cpp</code> on Zephyr 3.7 CAN API
Provisioning	thin host bootstrap + one-command demo; no sudo ever executed by scripts	<code>scripts/bootstrap-asi.sh</code> (west, SDK, CLI, board provisioning, SDK-store <code>idlc</code>), <code>scripts/run-tap-demo.sh</code> , <code>scripts/setup-tap.sh</code>

Table 1: Change surface by category. Vendored Autoware component logic: zero changes beyond the port’s adapter seams.

5 FVP integration

The Arm FVP_BaseR_AEMv8R (Cortex-R82, SMP-4) is the reference platform, and its integration is board-crate-driven rather than hand-glued:

- **Board bundle.** nano-ros ships `packages/boards/nros-board-zephyr/boards/fvp-aemv8r-smp/` with a `board.cmake` single source of truth: hardware-model-v2 board id `fvp_baser_aemv8r/fvp_aemv8r_aarch64/smp`, toolchain `aarch64-zephyr-elf`, default RMW CycloneDDS, Rust target `aarch64-unknown-none`, runner `armfvp`, plus the board’s Kconfig/DTS fragments and a `zephyr-rust-support` module. `nano_ros_use_board(fvp-aemv8r-smp)` before `find_package(Zephyr)` layers all of it in; `nros board info` prints the resolved contract.
- **NIC and networking.** The model’s smsc-91c111 NIC is off by default; `build.sh` enables it (`-C bp.smc_91c111.enabled=1`, promiscuous for the bridge path). Two profiles: FVP user-mode networking (DHCP; used by CI) and the TAP profile (`--network tap`: `-C bp.hostbridge.userNetworking=0` `-C bp.hostbridge.interfaceName=tap0`, DDS domain 2 pinned in the board conf) for the demo. IGMP is enabled for SPDP multicast join (fixed in the CycloneDDS fork during phase 3).
- **Run model.** `west build --target run` launches the FVP through Zephyr’s `armfvp` runner — no hand-rolled launch scripts. One model flag matters operationally: `-C cache_state_modelled=0`; with the default the model crawls $\approx 1000 \times$ under busy code, which presents as a “hang” at network bring-up.
- **CI.** A five-phase runtime suite downloads the sha-pinned FVP from Arm’s public CDN and runs: full controller boot to its liveness markers; the on-target unit suite; a DDS loopback; CAN output over Zephyr’s CAN loopback; and the TAP image build. The same script runs unchanged on a developer host (`.github/scripts/run-zephyr-fvp-ci.sh`).
- **Sizing.** The full image links to 9.7 MB of the FVP’s 128 MB RAM; the build-time knobs above bound the parameter pool, executor slots, and the subscription buffer (an 8.8 kB trajectory must fit a single buffer).

6 The 2026-08-20 modernization

The step advanced the nano-ros pin from `7dfe4fe4e` (2026-07-21) to `eace28852` (2026-08-20) — $\approx$ 2,400 upstream commits — and adopted the submodule layout. Eleven walls were logged (Table 2); none touched application logic. An earlier advance (July 2026) had surfaced eight walls, all fixed upstream within the cycle; notably, this jump produced *no* regression in any area an earlier wall had hardened.

#	Symptom	Root cause	Disposition
1	<code>nros setup board</code> rejects the board	CLI resolves the retired per-board crate path; bundle boards moved in an upstream refactor	Upstream issue #0729; steps inlined downstream
2	Configure fails: host <code>idlc</code> not found	Host CycloneDDS tooling moved to the CLI-managed SDK store	Provision via <code>nros setup <board> --rmw cyclonedds</code>
3	Board-facts rung silently absent	Facts resolver needs the nano-ros checkout path outside its own tree	<code>NR0S_REPO_DIR</code> exported; residual is #0729’s class
4	<code>nros-cpp</code> fails to compile (E0599)	Embedded C++ CycloneDDS feature set omits <code>alloc</code> while an error-mapper arm references an alloc-gated variant; masked upstream by <code>native_sim</code> ’s <code>std</code>	Upstream issue #0730; self-retiring downstream patch
5	Test programs: missing headers, unknown type names	Flat <code>idlc</code> -name compatibility layer retired; typed C++ API is now the only publisher/subscriber surface	Tests migrated onto the dual-mode message umbrella
6	<code>CAN_FILTER_DATA</code> undeclared	Zephyr 3.7 removed the flag; the CAN test had never been rebuilt on 3.7	Standard-ID filter is <code>flags = 0</code>
7	Entry panic policy changed	Upstream default moved from halt to the platform’s <code>k_panic()</code>	Accepted; loud fatal is preferable for a safety island
8	Sizing knobs went live	Previously inert Kconfig knobs are now forwarded; environment overrides Kconfig	Audited: build-time exports still authoritative
9	Boot markers never printed	Markers lived in the retired imperative <code>main.cpp</code> ; the generated entry prints no application banner	Component constructor now owns the markers
10	Tests: <code>create_node</code> returns <code>NOT_INIT</code>	Entry-less test images never initialized the runtime	Shim performs one-time <code>nros::init()</code>
11	Test hangs on node stop	<code>pthread_cancel</code> is deferred-only on Zephyr’s POSIX layer; the poll loop has no cancellation point	Cooperative stop flag replaces cancel

Table 2: Wall ledger for the $\approx$ 2,400-commit pin advance. Full details in the repository’s phase-3 roadmap document.

Three observations generalize. **Latent-validation debt surfaces on middleware moves**: walls 6, 9, 10, 11 were pre-existing gaps that the first complete runtime re-validation exposed, not regressions. **Coverage gaps are pairwise, not per-axis**: wall 4’s failing coordinate is the combination

(embedded Zephyr $\times$ C++ $\times$ CycloneDDS); each axis value was covered upstream, the pairing was not. **Consumer workarounds should be designed to die**: both downstream workarounds are idempotent scripts that detect the upstream fix and retire themselves.

7 Validation

Build. Six firmware modes from one branch: full controller, unit test, DDS loopback, CAN output, DDS publisher/subscriber.

Runtime. The five-phase FVP suite passes end to end (boot markers, `All Tests Passed`, DDS loopback, CAN output, TAP build).

Closed loop. With the stack of Figure 1 brought up headless (ego pose and goal seeded from the command line), the planner streams the trajectory at 10 Hz; the island’s MPC engages — from the spawn position it correctly commands an emergency stop on excessive tracking error — and publishes control commands back onto domain 1 at 19 Hz and above, matching the pre-modernization baseline. `scripts/run-tap-demo.sh` reproduces the run, including the rate check, in one command. Interoperability was additionally spot-checked at field level with `ros2 topic echo` on the Humble side.

8 Lessons for the ecosystem

1. **A reference consumer is cheap leverage for a middleware project** — eleven walls in one step, two genuine upstream bugs invisible to the project’s own CI.
2. **Pin in lockstep, and make the pin visible** — one submodule pointer plus one manifest revision, required to agree.
3. **Generated entries beat imperative boot code** — but application-level observability must then be owned by application components.
4. **Treat provisioning as data** — toolchains, RMW sources and patch sets come from nano-ros’s SDK index via idempotent commands; the consumer bootstrap stays thin across upstream refactors.
5. **Re-run everything on every move** — the costliest walls hid behind validation that had silently stopped running.

9 Status and future work

The Zephyr FVP target is fully migrated and validated on the modern nano-ros pin. Open items: S32Z270 board parity (upstream board bundle pending), moving the FreeRTOS targets onto nano-ros’s platform layer (retiring the vendored CycloneDDS entirely), adopting the launch-resolved entry spelling end-to-end, a long-duration soak, and upstreaming resolutions for issues #0729/#0730.

Sources: `docs/roadmap/phase-3-modern-nano-ros-migration.md` (wall ledger, runbook), `docs/roadmap/phase-1*.md`, `docs/design/workspace_mode.rst`, nano-ros book chapter *Importing a Board Crate*, nano-ros issues #0729/#0730. Repository: `github.com/newsrabntu/autoware-safety-island`, branch `nano-ros`.